

## Static view: Utilities

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer

<<Module>>

<<uses>>

oradio\_logging.py  
<<module>>

+ oradio\_log.debug()  
+ oradio\_log.warning()  
+ oradio\_log.info()  
+ oradio\_log.error()

system\_sounds.py  
<<module>>

+ play\_sound()

Every modules/classes can use these utilities

### Hardware Abstraction Layer

## Static view: Throttle

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer

<<uses>>

throttled\_monitor.py  
<<module>>

- throttled\_monitor: RpiThrottlingMonitor()

<<uses>>

throttled\_monitor.RpiThrottlingMonitor  
<<singleton>>

- stop\_event: Event()

- init()  
+ start()  
+ stop()  
+ run()  
+ force\_throttled()  
+ clear\_throttled()

The rpi-throttling-monitor thread  
check the throttle state in a loop

<<consists of>>

<<thread>>

rpi-throttling-monitor

+ stop\_event: Event()  
- vcgencmd  
- get\_throttle\_value()

### Hardware Abstraction Layer

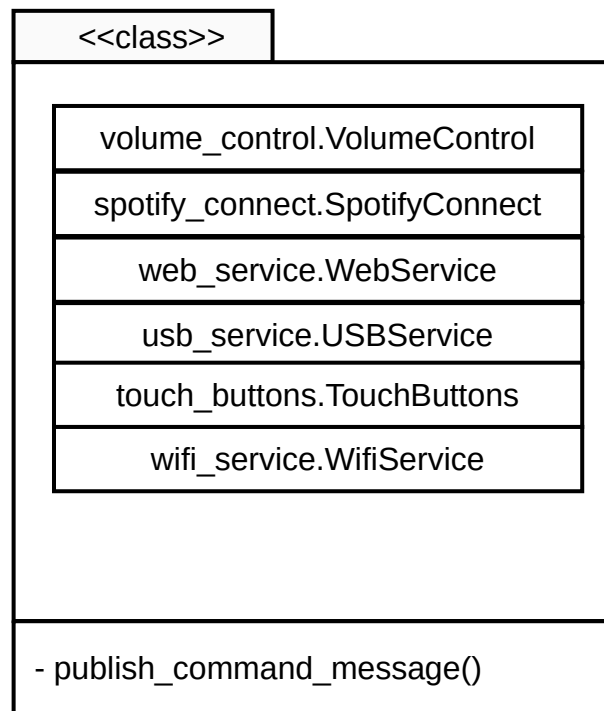
## Static view: Messages

### Application Layer

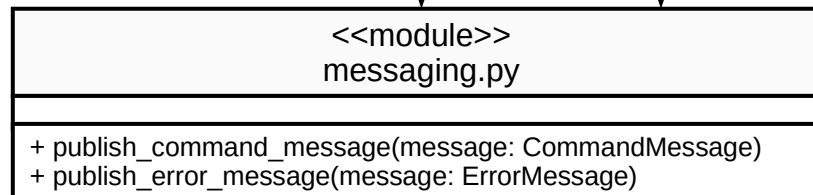
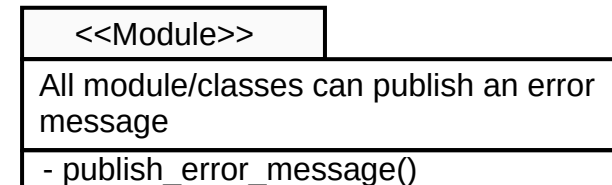
oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer



<<uses>>



### Hardware Abstraction Layer

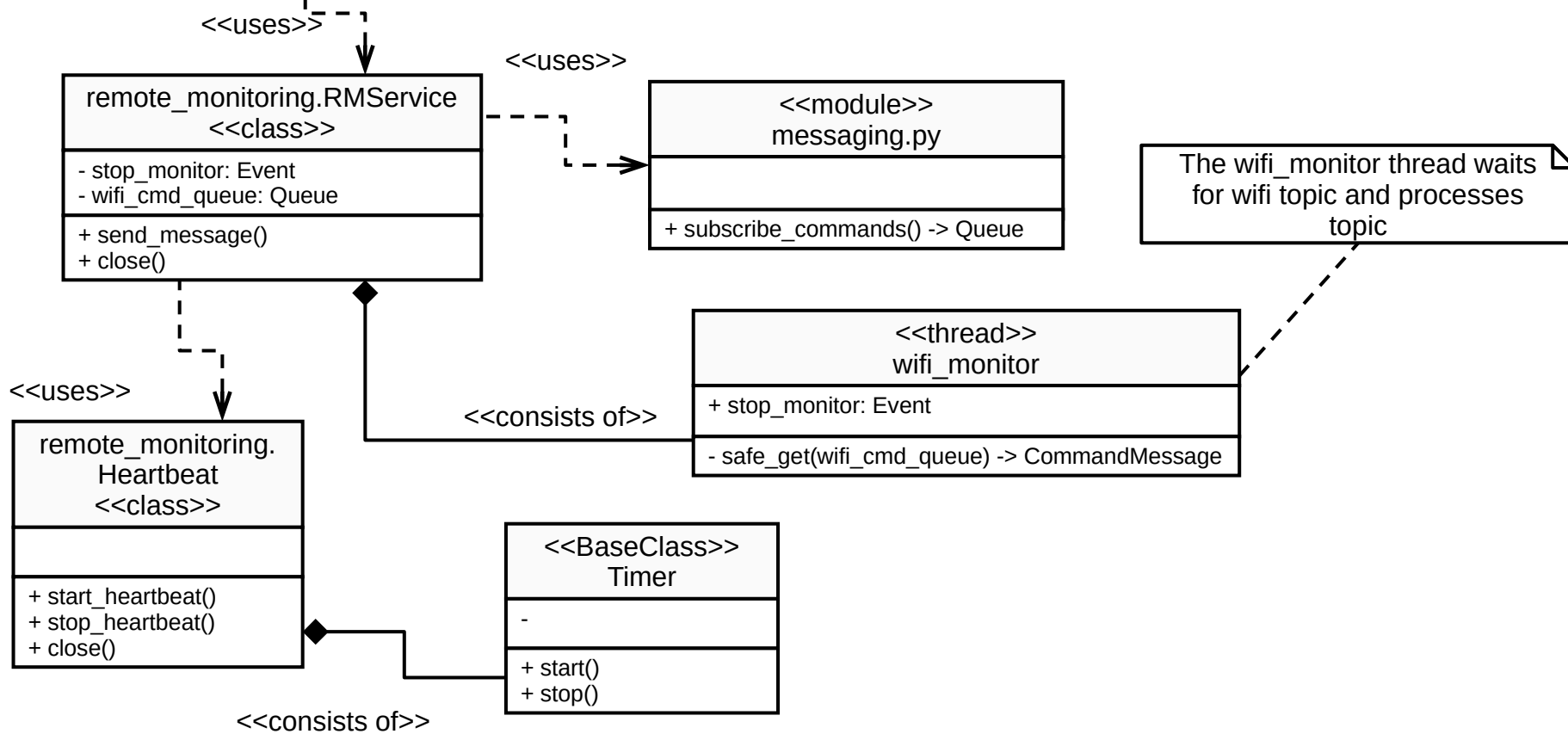
## Static view: Remote monitoring

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer



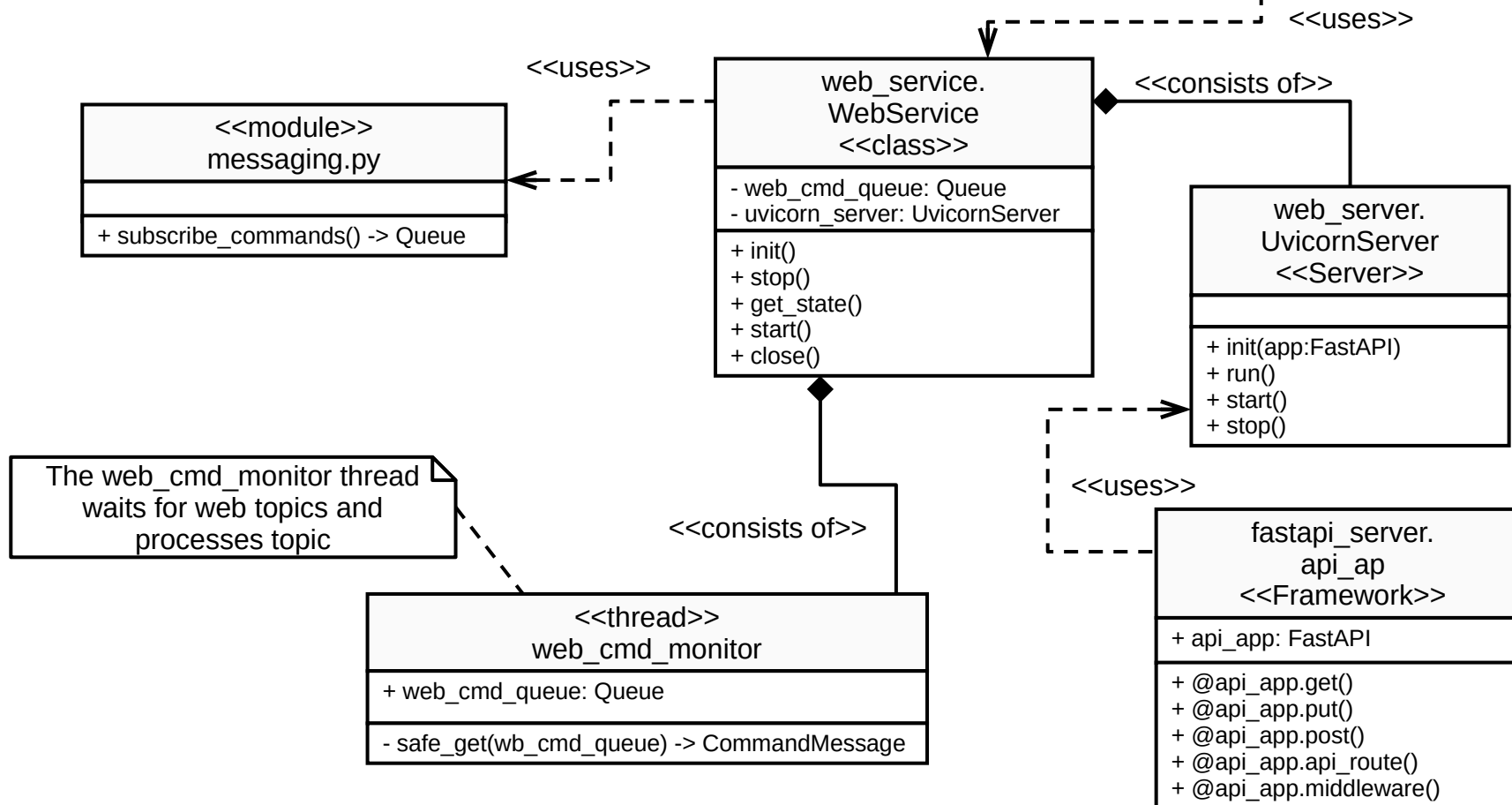
## Static view: Web Service

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer



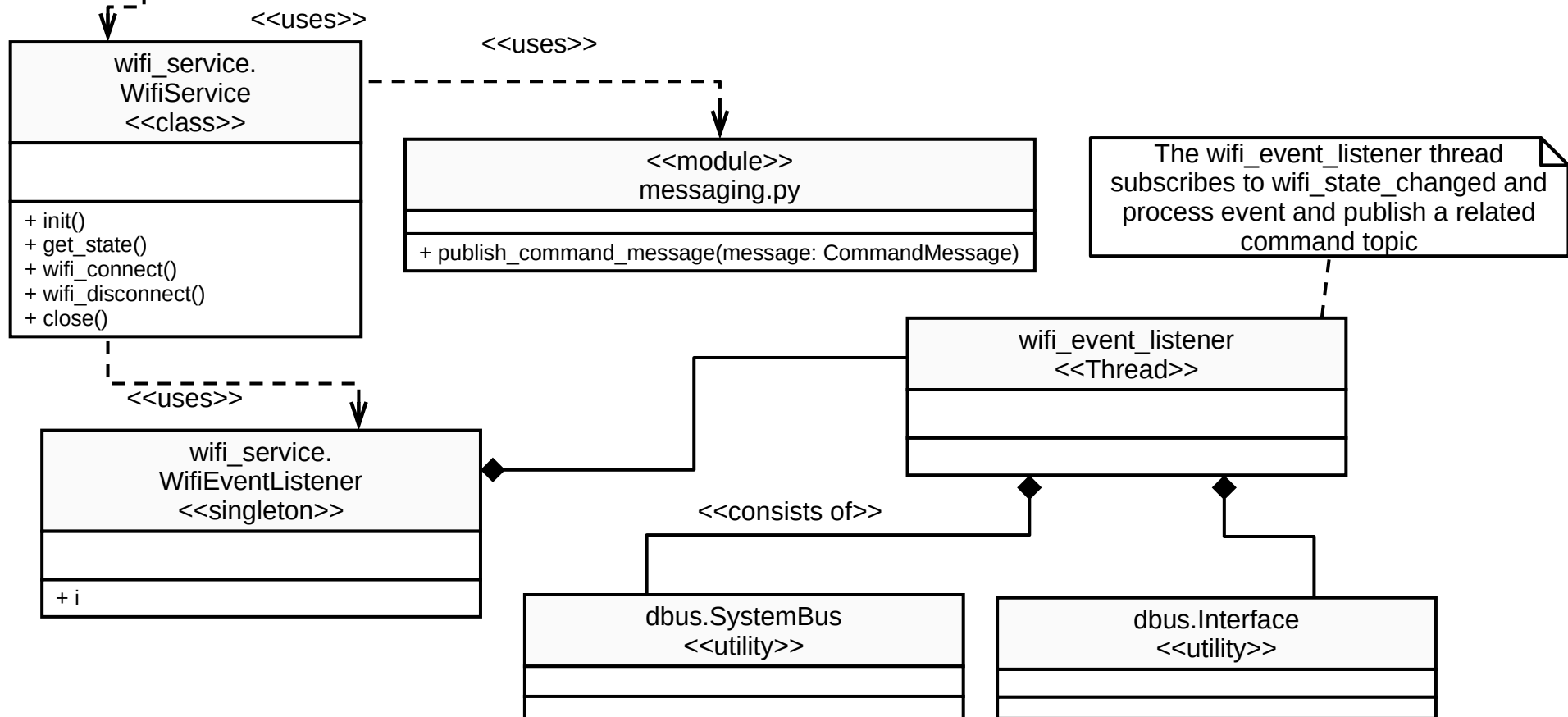
## Static view: WiFi Service

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer



## Static view: USB

### Application Layer

oradio\_control.py  
«module»

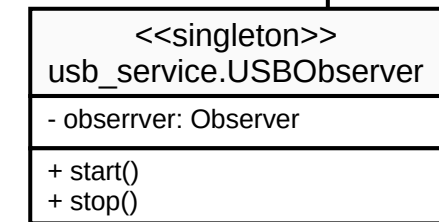
+ execute(): void

### Middle-ware Layer

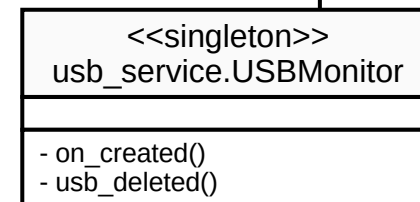
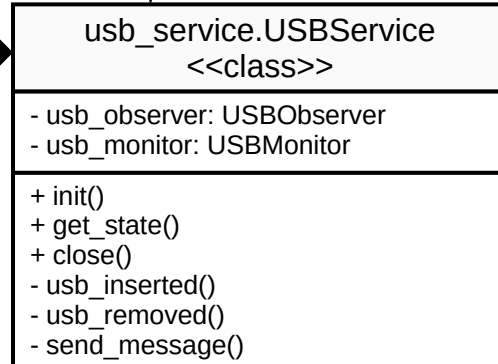
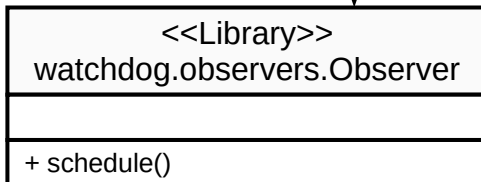
«uses»

«consists of»

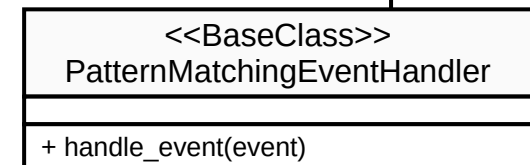
«consists of»



«uses»



«is a»



### Hardware Abstraction Layer

## Static view: Volume

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer

<<uses>>

Waarom start(), stop(), set\_notify().  
Staat altijd op notify. Dus notify default  
aan. Gaat nooit gestopt worden

volume\_control.VolumeControl  
<<class>>

+ set\_notify()  
+ start()  
+ stop()

<<has>>

i2c\_service.I2CService  
<<class>>

+ read\_byte(device: int, register: int)  
+ write\_byte(device: int, register: int)  
+ read\_block(device: int, register: int, length: int)  
+ write\_block(device: int, register: int, length: int)

### Hardware Abstraction Layer



## Static view: Error

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer

<<uses>>

error\_service.py.ErrorService  
<<class>>

- error\_queue:subscriber\_errors()

- init()  
- error\_handler()

<<thread>>

error\_service.error\_handler

<<uses>>

<<module>>  
messaging.py

+ ErrorMessage

+ safe\_get( error\_queue: Queue)

### Hardware Abstraction Layer

# Static view: Messaging

## Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

## Middle-ware Layer

<<uses>>

messaging  
<<module>>

- CommandMessage()  
- ErrorMessage()  
+ subscribe\_commands() -> Queue  
+ subscribe\_errors() -> Queue  
+ unsubscribe\_commands(queue: Queue)  
+ unsubscribe\_errors(queue: Queue)  
+ publish\_command(message: CommandMessage)  
+ publish\_error(message: ErrorMessage)  
+ safe\_get(queue: Queue)

<<has>>

messaging.PubSubManager  
<<singleton>>

- Topic.COMMAND: []  
- Topic.ERROR: []  
+ subscribe(topic: Topic) -> Queue  
+ unsubscribe(topic: Topic, queue: Queue)  
+ publish(topic: Topic, message: CommandMessage | ErrorMessage)

<<uses>>

messaging.Topic  
<<data>>

+ COMMAND = "COMMAND"  
+ ERROR = "ERROR"

messaging.ErrorMessage  
<<data>>

+ source: String  
+ message: String  
+ is\_valid()

messaging.CommandMessage  
<<data>>

+ source: String  
+ message: String  
+ data: Any  
+ is\_valid()

## Hardware Abstraction Layer

## Static view: Power Supply

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer

<<uses>>

power\_supply\_control.PowerSupplyService  
<<class>>

+ set\_nom\_voltage()  
+ set\_max\_voltage()  
+ set\_standby\_voltage()  
+ read\_status()

<<has>>

<<singleton>>

i2c\_service.I2CService

+ read\_byte(device: int, register: int)  
+ write\_byte(device: int, register: int)  
+ read\_block(device: int, register: int, length: int)  
+ write\_block(device: int, register: int, length: int)

### Hardware Abstraction Layer

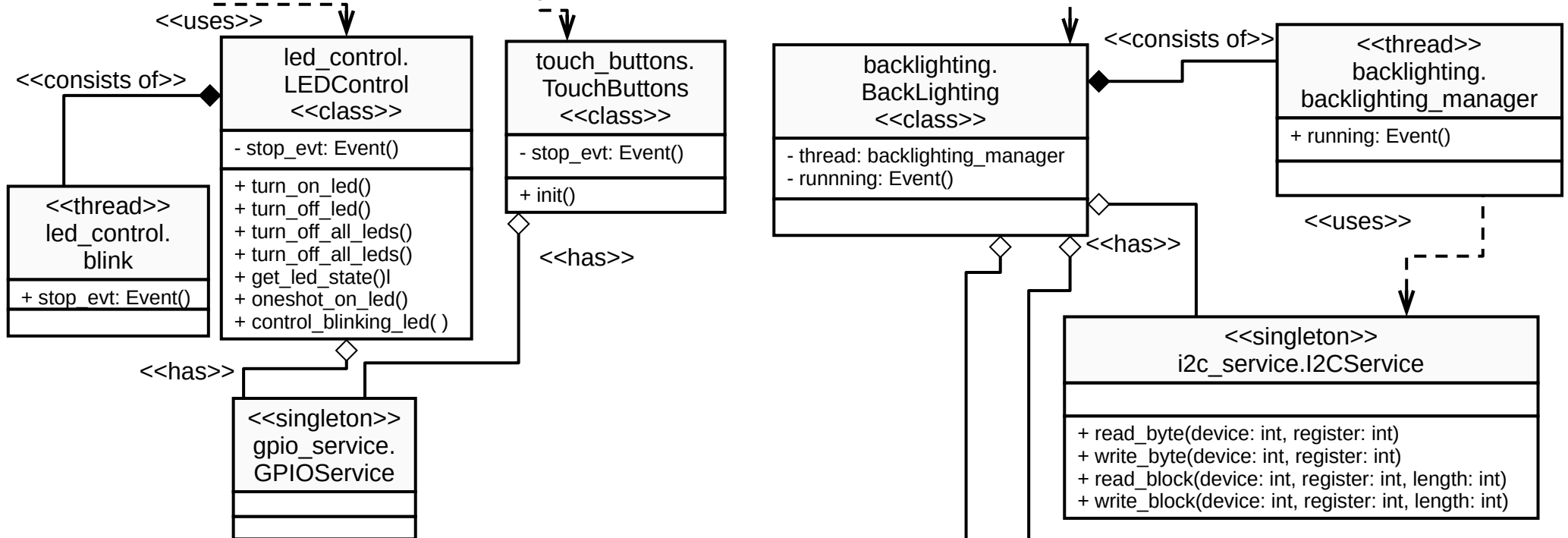
## static view: Front-panel

### Application Layer

oradio\_control.py  
<<module>>

+ execute(): void

### Middle-ware Layer



### Hardware Abstraction Layer

Waarom backlighting thread met stop en off. Staat eigenlijk altijd aan! Wordt ook in de module gestart!

# Static View: Music Players

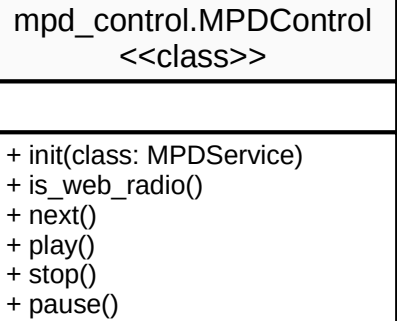
## Application Layer

oradio\_control.py  
<<module>>

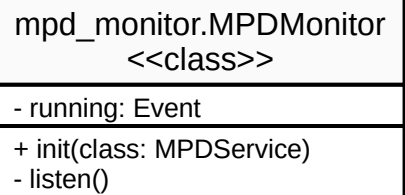
+ execute(): void

## Middle-ware Layer

<<uses>>



<<uses>>



<<thread>>  
mpd\_monitor.  
listen

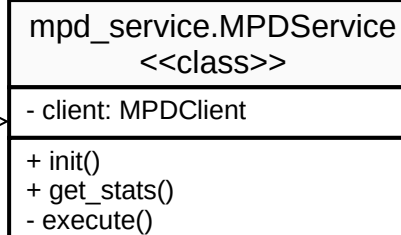
+ running: Event

<<uses>>

spotify\_connect.SpotifyConnect  
<<class>>

+ play()  
+ pause()  
+ get\_state()

<<is A>>



mpd.MPDClient  
<<library>>

+ init()  
+ idle()  
+ update()  
+ next()  
+ play()  
+ stop()  
+ pause()  
+ status()

<<consists of>>

<<is A>>

## Hardware Abstraction Layer